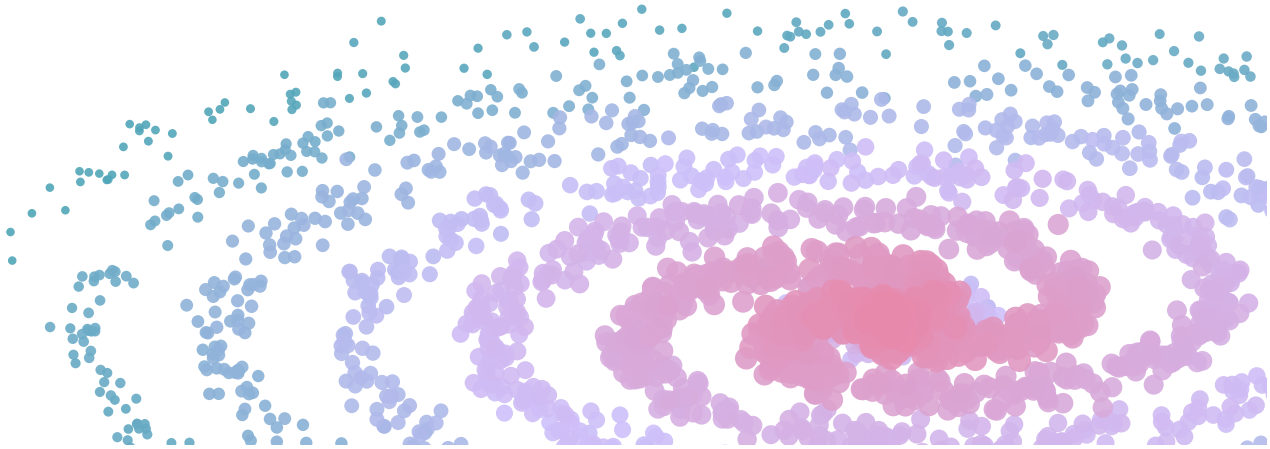
Generelalized time series forecasting using event streams pretrained LLM



Recasting Temporal Prediction as Next-Event Modeling with Pretrained LLMs

AUTHOR
Evgeny Adylin

AFFILIATION
Independent Research

PUBLISHED
April 28, 2026

1. Introduction

Time series forecasting is a central problem in many real-world applications, including electricity markets, finance, logistics, and industrial systems. At the same time, there is growing interest in forecasting not only numerical time series, but also sequences of events drawn from a finite alphabet, such as categorical state transitions, symbolic observations, transaction streams, and other discrete temporal processes. This broader perspective

suggests that forecasting can be viewed not only as prediction in continuous numeric space, but also as sequence modeling over event streams.

Most conventional forecasting pipelines operate directly in the numeric domain and rely either on classical statistical methods or on neural architectures explicitly designed for regression. In contrast, recent advances in large language models motivate a more general view of forecasting as next-event prediction over structured token sequences. From this perspective, temporal data can be represented as event streams, and forecasting becomes the task of modeling their autoregressive continuation using pretrained sequence models.

Such a formulation is attractive for several reasons. First, it provides a unified framework for modeling heterogeneous temporal processes, including both continuous-valued observations and discrete event sequences. Second, it enables the use of the standard next-token prediction objective that underlies pretrained LLMs, without requiring specialized forecasting architectures. Third, it creates a direct conceptual bridge between time series analysis and sequence generation, making it possible to transfer ideas, architectures, and training strategies from language modeling to temporal prediction.

In this work, we study this generalized forecasting perspective through a concrete case study based on a univariate time series. Specifically, a numerical electricity-price series is transformed into a sequence of discrete events, allowing the forecasting problem to be reformulated as autoregressive token prediction. A GPT-style language model is then used to model this event stream and generate future tokens, which are subsequently decoded back into the original numeric domain. In this setting, the discretized time series serves not as the main conceptual object, but as one practical example of how continuous temporal dynamics can be represented within an event-stream forecasting framework.

The objective of this article is to present the full forecasting pipeline, describe the dataset quantitatively and semantically, explain the event-based representation and decoding procedures, and analyze the forecasting experiment implemented in code. Using a synthetic daily electricity price dataset as a case study, we investigate whether a compact GPT-style model can learn meaningful temporal structure from a custom tokenized representation of temporal dynamics, and how this approach can contribute to the broader problem of generalized forecasting with event streams and pretrained language models.

2. Method Overview

As an illustrative example of the proposed general event-stream forecasting framework, this study considers a symbolic event sequence derived from a numerical time series. Specifically, the original univariate series is transformed into an event stream by computing first-order differences and then quantizing these differences into a finite set of discrete states. This construction allows a standard numerical forecasting problem to be reformulated as prediction over a finite event alphabet. The full forecasting pipeline consists of the following stages:

The method is based on the idea that the transformer does not need to operate directly on continuous values. Instead, it can learn temporal structure from a symbolic representation of local changes in the signal.

The core design decision is to represent the time series through quantized first-order differences rather than through raw values. This makes the representation more stable and naturally aligned with local dynamics.

The figure above shows the complete daily electricity price series used in the study. The data exhibits long-range structure, changing level, and medium-scale fluctuations, which makes it suitable for studying symbolic autoregressive forecasting.

3. Tokenizer Design

3.1 Difference-based representation

Let the original time series be a sequence of values x(1), x(2), ..., x(T). Instead of modeling these values directly, the method first transforms the series into first-order differences:

Delta x(t) = x(t) - x(t-1)

This representation shifts the problem from absolute levels to local dynamics. It reduces sensitivity to long-range drift and makes the distribution more compact and easier to discretize.

3.2 Quantization into bins

The difference sequence is quantized into bins using a train-only fitting procedure. The implementation uses:

- `N_BINS = 65`
- `USE_OUTLIERS = True`
- `Q_LOW = 0.25`
- `Q_HIGH = 0.75`
- `IQR_K = 1.5`

The inlier region is split into 65 bins, while two additional bins are reserved for extreme negative and extreme positive outliers. The final token space therefore includes `bin_0` to `bin_66`.

The outlier boundaries are estimated using the interquartile range rule on the training split only:

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

For the current dataset, these fitted train-only boundaries are approximately:

- lower bound: **-7.210**
- upper bound: **7.179**

The histogram above shows the distribution of first-order differences and the fitted outlier thresholds. This figure is central to the tokenizer design, because it illustrates how the continuous difference space is partitioned into symbolic bins.

3.3 Reconstruction from bin means

After binning, each token can be mapped back to a representative numerical value using the mean difference associated with the corresponding bin, estimated on the training set only.

This allows the original series to be approximately reconstructed from the discretized representation. The reconstruction RMSE for the current configuration is **4.061**, which measures the information loss introduced by quantization alone.

These figures show that the quantized representation preserves much of the global structure of the series, although some fine-grained variation is naturally smoothed by the binning process.

3.4 Vocabulary construction

The tokenizer is implemented as a custom WordLevel tokenizer. Its vocabulary contains:

- special tokens: `[PAD]`, `[BOS]`, `[EOS]`, `[UNK]`
- a start token: `start`
- bin tokens: `bin_0` to `bin_66`

The tokenizer is built from training data only. In the current experiment, the resulting vocabulary size is **72** tokens.

4. Language Modeling Formulation

Once the difference sequence has been quantized, the original time series is transformed into a token stream of the form:

```
start bin_37 bin_42 bin_41 ...
```

This sequence is then treated as ordinary autoregressive text.

4.1 Token sequence construction

The token stream is converted into token IDs and chunked into fixed-length blocks for causal language modeling. The implementation uses:

- `BLOCK_SIZE = 128`

The current dataset yields:

- **42 training blocks**
- **15 validation blocks**

Training blocks are formed with `drop_remainder=True`, while validation blocks retain the final incomplete segment and pad it if needed.

4.2 GPT-2 architecture

A compact GPT-2 model is initialized from scratch with the following configuration:

- context length: `128`
- embedding size: `384`
- number of layers: `6`
- number of attention heads: `6`
- residual dropout: `0.2`
- embedding dropout: `0.2`
- attention dropout: `0.2`

This results in a model with approximately **10.7 million parameters**.

4.3 Decoding generated tokens

During inference, the model generates future tokens autoregressively. These tokens are mapped back to bin indices, then converted to numerical differences using train-only bin means.

Starting from the last observed value x(T), the forecast is reconstructed recursively:

x_hat(T+1) = x(T) + mean_diff(predicted_bin_1)

x_hat(T+h) = x_hat(T+h-1) + mean_diff(predicted_bin_h)

This converts a token-generation process back into a fully numerical forecasting pipeline.

5. Experimental Setup

5.1 Dataset

The experiments are conducted on `electricity_price_daily_synthetic.csv`, a univariate daily time series with two columns: `date` and `electricity_price`.

The dataset contains **7,275 observations** covering the period from **2005-01-31** to **2024-12-31**. The series has no missing calendar dates.

Basic statistics of the target series:

- mean: **92.12**
- median: **91.30**
- standard deviation: **20.42**
- minimum: **30.98**
- maximum: **145.60**

5.2 Train / validation split

The data is split chronologically using a **75/25** split:

- training set: **5,456 observations**
- validation set: **1,819 observations**

All data-dependent preprocessing steps are fitted on the training portion only, including:

- bin boundaries,
- outlier thresholds,
- token vocabulary,
- bin means used for decoding.

5.3 Training configuration

The GPT-2 model is trained using the Hugging Face `Trainer` API with a causal language modeling objective.

The main configuration is:

- epochs: **80**
- train batch size: **8**
- validation batch size: **8**

- learning rate: **1e-6**
- warmup steps: **30**
- weight decay: **0.01**
- early stopping patience: **3**
- evaluation every **4** steps
- logging every **4** steps

5.4 Forecasting configuration

The active short-horizon forecasting setup uses:

- context window: **20**
- horizon: **10**
- `top_k = 30`
- `top_p = 0.9`
- `temperature = 0.7`

5.5 Baselines and metrics

The method is compared against:

- `NaiveSeasonal(K=10)`
- `Theta(theta=2)`
- `ExponentialSmoothing`

Evaluation is performed using:

- MAE
- RMSE

6. Results

6.1 Quantization and reconstruction quality

Before evaluating forecasting performance, it is useful to separate the approximation error introduced by quantization from the forecasting error introduced by the model itself.

When the original series is reconstructed from train-only bin means, the resulting reconstruction RMSE is:

- **4.061**

This confirms that the symbolic representation preserves a substantial amount of the original structure, while still compressing the signal into a discrete token space.

6.2 Training dynamics

The training process is stable and shows consistent improvement over time. Both the training loss and the validation loss decrease throughout optimization, indicating that the tokenized sequence is learnable within the causal language modeling setup.

The training loss decreases from **4.321111** at step **4** to **4.201290** at step **480**. The validation loss decreases from **4.316453** at step **4** to **4.195526** at step **480**. The best validation loss recorded in this run is **4.195526**, reached at step **480**.

The relatively small gap between the two curves suggests that optimization remains stable and does not exhibit strong overfitting within the observed training horizon.

6.3 Forecast examples and baseline comparison

The forecasting experiment is evaluated on a horizon of **10** future steps starting from the boundary between the training and validation segments. The figures below illustrate both the standalone GPT-2 forecast and its comparison with classical baseline methods.

The first figure shows the direct GPT-2 forecast against the true future trajectory. The model captures the general direction of the continuation and follows the medium-scale dynamics of the series, although it smooths some of the sharper local movements. For this forecast window, the GPT-2 model achieves:

- **MAE = 2.2138**
- **RMSE = 2.6956**

The comparison with baseline methods provides a clearer view of the relative strengths of the proposed approach. Quantitatively, the results are:

Model	MAE	RMSE
GPT-2	2.2138	2.6956
Naive	2.1145	2.8350
Theta	2.5750	3.1319
ExponentialSmoothing	3.3063	3.8719

These results show that the GPT-2-based token forecasting model achieves the **best RMSE** among the evaluated methods, indicating the strongest performance in terms of squared prediction error on this horizon. The Naive baseline obtains a slightly lower MAE, which suggests that simple seasonal repetition remains competitive in average absolute deviation on this particular forecast window. However, GPT-2 produces a lower RMSE than Naive, which indicates fewer large deviations overall.

The comparison against Theta and ExponentialSmoothing is more clearly favorable to the proposed method. GPT-2 outperforms both baselines on both MAE and RMSE, suggesting that the tokenized autoregressive formulation is capable of modeling short-horizon local dynamics more effectively than these classical alternatives in the current setup.

Overall, the empirical evidence supports two conclusions. First, the tokenizer and language-modeling formulation are fully operational and produce meaningful forecasts in the original

numerical domain. Second, the proposed GPT-2-based approach is already competitive with standard forecasting baselines, especially when evaluated through RMSE, which is more sensitive to larger forecast errors.

7. Discussion

The main contribution of this work lies in the representation rather than in the backbone architecture. The experiments show that a univariate time series can be transformed into a symbolic sequence and modeled with a standard GPT-2 language model without changing the transformer objective.

The most important design choice is to tokenize first-order differences instead of absolute values. This representation is better aligned with local dynamics and produces a compact distribution that can be quantized effectively.

At the same time, quantization introduces irreversible information loss. The reconstruction plots show that the major structure of the series is preserved, but fine-scale variability is partially smoothed. This trade-off is central to the method: the tokenizer makes the sequence learnable as text, but at the cost of precision.

Another strength of the implementation is the strict train-only fitting of the tokenizer and decoder statistics. This prevents leakage and makes the experimental design methodologically sound.

Future improvements should investigate:

- different numbers of bins,
- alternative discretization strategies,
- comparison between tokenized differences and tokenized levels,
- rolling-origin evaluation over multiple forecast windows,
- application to real-world electricity market data.

8. Conclusion

This article presented a method for univariate time series forecasting based on custom tokenization and autoregressive language modeling. The proposed pipeline transforms first-order differences into quantized bins, maps them to discrete tokens, and trains a GPT-2 model from scratch on the resulting sequence.

The resulting system demonstrates that forecasting can be reformulated as next-token prediction while still allowing full decoding back into the numerical domain. In this formulation, the tokenizer is the central component of the method, because it defines the symbolic language through which the model perceives time-series dynamics.

The current implementation already includes:

- train-only quantization,
- custom tokenization,
- GPT-2 training,
- interactive loss visualization,
- reconstruction analysis,
- comparison setup against classical baselines.

The next stage of the work is to complete the forecasting evaluation with numerical comparisons and trajectory plots, and then extend the study to broader datasets and more extensive backtesting.

Citation For attribution in academic contexts, please cite this work as

```
Evgeny
Adylin
(2026).
"Generelaliz
ed time
series
forecasting
using event
streams
pretrained
LLM".
```

BibTeX citation

```
@misc{adylin
2026_generel
alized_time_
series_forec
asting_using
_event_strea
ms_pretraine
d_llm,
  title=
{Generelaliz
ed time
series
forecasting
using event
streams
pretrained
LLM},
  author=
{Evgeny
Adylin},
  year=
{2026},

}
```

Made with ❤️ with research article template